

Intermediate (apply the follow-up template yourself) — Solution

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Scenario question: "Your RAG agent's retriever is returning irrelevant chunks in production, but it looked fine in testing. Walk me through how you'd debug it."

Basic Version

First answer: "I'd first check whether it's really the retriever, or something downstream that just looks like a retrieval problem. I'd take a few of the bad answers, look at exactly which chunks got pulled back, and see if those chunks make sense for the question or not."

Follow-up — what's underneath: "If the chunks really are wrong, it's usually one of three things: the documents changed and the search index is out of date, the way we're splitting documents into chunks is cutting sentences in a bad spot, or the questions coming in now look different from the ones we tested with."

Follow-up — how would you debug it: "I'd run the exact same test questions we used before, against the live system, and see if those still work. If they do, it's the new real-world questions that are the problem, not the retriever itself."

Follow-up — what happens if it fails again: "Right now, probably nothing — it just quietly gives a worse answer. I'd want a low-confidence flag so the system can say 'not sure' instead of guessing."

This is a complete, honest, plain-English answer — what a nervous first-timer says when they genuinely know the material but haven't drilled the polished vocabulary yet. It answers every follow-up without stalling and shows real problem-solving order (isolate, then diagnose, then think about failure).

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Intermediate Version

First answer: "I'd isolate the layer first — pull the actual retrieved chunks for a handful of the bad live queries and check relevance directly, before assuming it's the retriever at all, since a prompt or generation regression can look identical to a retrieval regression from the outside."

Follow-up — what's underneath: "Assuming the chunks themselves are genuinely off-topic, the three usual suspects are: index staleness — the underlying documents changed and weren't re-embedded; a chunking regression — a recent change to chunk size or boundaries is now splitting semantic units mid-sentence, hurting embedding quality; or distribution shift — the live query distribution has drifted from what the retriever was tuned and tested against."

Follow-up — how would you debug it: "I'd re-run the original test suite's queries against the live index first — if those still score well, the retriever itself is fine and the problem is upstream (new query patterns we never tested). If the suite now fails too, I'd bisect: check the index's last update timestamp against the last chunking-logic deploy, since that pinpoints which of the two most likely causes it is."

Follow-up — what happens if it fails again: "Right now this fails silently — a bad retrieval still produces a fluent, plausible-sounding wrong answer, which is the worst kind of failure because nothing looks broken. I'd add a similarity-score threshold: below it, the system says it's not confident instead of generating from weak context."

Why this answer works: it's this document's Normal and Deep rungs, done properly, stacked on top of the standing follow-up list — it starts by isolating the failing layer instead of assuming the premise ("the retriever") is even correct, which is a senior instinct, not a junior one. Each follow-up answer names a *specific, checkable mechanism* (index staleness, chunking regression, distribution shift) instead of a vague category. It treats "what happens if it fails again" as a design gap to fix (a confidence threshold), not just a fact to report.

What the Basic Version misses: the Basic Version's answers are correct but stop at the category level ("the docs changed," "the chunking's off") — the Intermediate Version names the *specific mechanism* behind each category (re-embedding lag, mid-sentence chunk boundaries hurting embedding quality, distribution shift specifically vs. testing distribution) and connects the debug step to a concrete before/after signal, not just "I'd look into it."

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Advanced Version

The pushback, right after the Intermediate answer's confidence-threshold fix: *"That threshold is going to make the system refuse to answer a lot of borderline-but-actually-fine queries. How do you know where to set it, and what does the product team say when 'I'm not sure' shows up for 15% of traffic?"*

Model response, defending the fix without dodging the real cost: "That's a fair cost, and I wouldn't pick the threshold by guessing — I'd calibrate it against a labeled validation set: take a sample of past queries with known-good and known-bad retrievals, plot similarity score against actual correctness, and pick the cutoff that hits a target false-refusal rate the product team signs off on, not a number I chose by feel. And I'd track that refusal rate as its own metric post-launch, because the right threshold at launch can drift as the corpus and query mix change — the same distribution-shift risk that caused this incident in the first place.

On the 15% number specifically: I'd push back gently on whether that's actually the finding, versus a worst-case estimate — but if it really is that high, the comparison that matters isn't 'refusal rate versus zero,' it's 'refusal rate versus the rate of confident-but-wrong answers we're shipping today without the threshold.' For most products, a system that visibly says 'I'm not sure' is a better user experience than one that answers wrong fluently and erodes trust once someone catches it — but I'd name the exception: if this is a low-stakes, high-volume use case where a wrong answer costs almost nothing and a refusal actively breaks the user flow, a lower-confidence answer with a caveat might beat an outright refusal, and that's a product call, not a purely technical one."

Why this answer works: it doesn't retreat from the fix just because a real cost was named, and it doesn't pretend the cost isn't real either — it answers the actual question asked ("how do you know where to set it") with a concrete calibration method instead of restating that thresholds are good practice. It reframes the pushback's implicit comparison (refusal rate vs. zero) to the comparison that actually matters (refusal rate vs. today's silent wrong-answer rate), which is the kind of reframe that shows the candidate is thinking about the system, not just defending a talking point. And it names the one case where the interviewer's pushback would actually win — showing judgment instead of blind defense.

What a weaker answer misses: a weaker answer either caves immediately ("okay, maybe the threshold isn't a good idea then") — abandoning a genuinely sound fix the moment it's challenged, which reads as not having thought it through in the first place — or gets defensive and repeats "well, it's still better than nothing" without engaging the specific number the interviewer raised, which reads as not actually listening to the pushback.

Which one should you actually give in a real interview? Lead with the Basic Version's plain framing so a non-specialist interviewer can follow it, then be ready to drop into the Intermediate Version's specific mechanisms (index staleness, chunking regression, distribution shift, similarity threshold) the moment you're asked "how do you know" or "what specifically." Save the Advanced Version's defend-under-pushback move for when it's actually needed — don't pre-empt a challenge nobody raised, since that reads as arguing with yourself. But when the pushback does land, answer it the way the Advanced Version does: name the real cost, don't dodge it, and say specifically what would change your mind.